



Experiment No. 3

**Title: Implementation of Stemming of Text using
different stemming modules in NLTK**



Batch: NLP-1

Experiment No.: 03

Roll No.: 16010422234

Name: Chandana Galgali

Aim of the Experiment: To implement stemming of Text using different stemming modules in NLTK

Program/Steps:

1. Apply Porter Stemmer, Snowball Stemmer, Lancaster Stemmer and Regexp Stemmer to a large corpus of Text.
2. Write down the observations about differences in the working of each of the Stemmer by analyzing the result obtained after applying each Stemmer on large corpus of text

```
import sys
import subprocess
import os
import re
from collections import Counter, defaultdict

# --- Helper: install missing packages ---
def ensure_pkg(pkg_name: str):
    try:
        __import__(pkg_name)
    except ImportError:
        subprocess.check_call([sys.executable, "-m", "pip", "install",
pkg_name])
        __import__(pkg_name)

ensure_pkg("nltk")
ensure_pkg("pandas")

import nltk
import pandas as pd

# --- Step 1: Download required NLTK resources ---
def ensure_nltk_data():
    resources = [
        ("corpora", "gutenberg"),
        ("corpora", "brown"),
        ("corpora", "reuters"),
        ("corpora", "stopwords"),
]
```

```

for rtype, name in resources:
    try:
        nltk.data.find(f"{rtype}/{name}")
    except LookupError:
        nltk.download(name)

ensure_nltk_data()

from nltk.corpus import gutenbergl, brown, reuters, stopwords
from nltk.stem import PorterStemmer, SnowballStemmer, LancasterStemmer,
RegexpStemmer

# --- Step 2: Build a large corpus (Gutenberg + Brown + Reuters) ---
def build_large_corpus_text() -> str:
    texts = []
    # Gutenberg
    for fid in gutenbergl.fileids():
        texts.append(gutenbergl.raw(fid))
    # Brown (join words per category)
    for cat in brown.categories():
        words = brown.words(categories=cat)
        texts.append(" ".join(words))
    # Reuters (join words per file id)
    for fid in reuters.fileids():
        words = reuters.words(fid)
        texts.append(" ".join(words))
    return "\n".join(texts)

# --- Step 3: Preprocess (lowercase, alphabetic tokens, remove stopwords)
---
def preprocess(text: str):
    # simple regex tokenizer: keep only alphabetic sequences
    tokens = re.findall(r"[A-Za-z]+", text.lower())
    sw = set(stopwords.words("english"))
    tokens = [t for t in tokens if t not in sw and len(t) > 2]
    return tokens

# --- Step 4: Apply stemmers ---
def stem_with_all(tokens):
    porter = PorterStemmer()
    snowball = SnowballStemmer(language="english") # ignore_stopwords not
needed after removing them
    lancaster = LancasterStemmer()

```

```

regexp = RegexpStemmer('ing$|s$|e$|able$', min=4)

stems = {
    "porter": [porter.stem(t) for t in tokens],
    "snowball": [snowball.stem(t) for t in tokens],
    "lancaster": [lancaster.stem(t) for t in tokens],
    "regexp": [regexp.stem(t) for t in tokens],
}

return stems

# --- Utility: Summaries ---
def summarize(tokens, stems_dict):
    vocab_original = set(tokens)
    print("\n=== SIZE SUMMARY ===")
    print(f"Total tokens (after preprocessing): {len(tokens):,}")
    print(f"Unique tokens BEFORE stemming: {len(vocab_original):,}")

    rows = []
    for name, stems in stems_dict.items():
        vocab_stem = set(stems)
        reduction = 100.0 * (1 - (len(vocab_stem) / len(vocab_original)))
        rows.append([name, len(vocab_stem), f"{reduction:.2f}%"])
    df = pd.DataFrame(rows, columns=["Stemmer", "Unique tokens AFTER",
    "Vocab reduction vs. original"])
    print(df.to_string(index=False))
    return df

# --- Utility: Word → stems side-by-side table ---
def build_word_to_stems_table(tokens, stems_dict, max_words=5000):
    # We'll create a mapping for the most common words (to keep file size
    reasonable)

    freq = Counter(tokens)
    most_common_words = [w for w, _ in freq.most_common(max_words)]

    records = []
    for w in most_common_words:
        rec = {"word": w}
        for name, stems in stems_dict.items():
            # find the first occurrence of w and pick its stem (consistent
            for same word)
            # more efficient: compute once a word->stem map for each
            stemmer

            rec[name] = None

```

```

        records.append(rec)

    # Precompute per-stemmer word->stem maps
    maps = {}
    for name, stems in stems_dict.items():
        mapping = {}
        for tok, st in zip(tokens, stems):
            if tok not in mapping:
                mapping[tok] = st
        maps[name] = mapping

    for rec in records:
        w = rec["word"]
        for name, mapping in maps.items():
            rec[name] = mapping.get(w, "")

    df = pd.DataFrame.from_records(records, columns=["word", "porter",
"snowball", "lancaster", "regex"])
    return df

# --- Utility: Collisions (potential over-stemming evidence) ---
def find_collisions(tokens, stems):
    # Map stem -> set(words) that produced it
    coll = defaultdict(set)
    for w, s in zip(tokens, stems):
        coll[s].add(w)

    # Keep stems that come from 2+ different words
    collisions = {stem: ws for stem, ws in coll.items() if len(ws) >= 2}
    # Rank by how many distinct words collapsed
    ranked = sorted(collisions.items(), key=lambda kv: len(kv[1]),
reverse=True)
    return ranked

def collisions_to_dataframe(ranked, topn=50, stemmer_name=""):
    rows = []
    for stem, words in ranked[:topn]:
        # show up to 12 originals to keep row readable
        sample = ", ".join(list(sorted(words))[:12])
        rows.append([stemmer_name, stem, len(words), sample])
    df = pd.DataFrame(rows, columns=["stemmer", "stem",
"distinct_original_words", "originals_sample"])
    return df

```

```

# --- Utility: Hand-checked pairs to illustrate under/over stemming ---
def inspect_examples(stem_fns):
    pairs_over = [
        ("universe", "university"),
        ("policy", "police"),
        ("general", "generation"),
        ("final", "finally"),
        ("organ", "organic"),
    ]
    pairs_under = [
        ("analysis", "analyst"),
        ("connect", "connection"),
        ("biology", "biological"),
        ("economy", "economic"),
        ("happy", "happiness"),
    ]
    print("\n=== CHECKED EXAMPLES (POTENTIAL OVER-STEMMING) ===")
    for a, b in pairs_over:
        row = [f"{a:<12} / {b:<12}"]
        for name, fn in stem_fns.items():
            row.append(f"{name}: {fn(a)} / {fn(b)}")
        print(" | ".join(row))

    print("\n=== CHECKED EXAMPLES (POTENTIAL UNDER-STEMMING) ===")
    for a, b in pairs_under:
        row = [f"{a:<12} / {b:<12}"]
        for name, fn in stem_fns.items():
            row.append(f"{name}: {fn(a)} / {fn(b)}")
        print(" | ".join(row))

def main():
    out_dir = "stemming_outputs"
    os.makedirs(out_dir, exist_ok=True)

    print("Building large corpus from Gutenberg + Brown + Reuters...")
    text = build_large_corpus_text()
    print("Preprocessing (lowercase, alphabetic only, remove stopwords,
len>2) ...")
    tokens = preprocess(text)

    print("Applying stemmers...")
    stems_dict = stem_with_all(tokens)

```

```

# Summary metrics
df_summary = summarize(tokens, stems_dict)
df_summary.to_csv(os.path.join(out_dir, "summary_metrics.csv"),
index=False)

# Side-by-side word -> stems
df_map = build_word_to_stems_table(tokens, stems_dict, max_words=5000)
df_map.to_csv(os.path.join(out_dir, "word_to_stems_5k.csv"),
index=False)
print(f"\nSaved word-to-stems sample table to {os.path.join(out_dir,
'word_to_stems_5k.csv')}")

# Collisions for each stemmer
all_collisions = []
for name, stems in stems_dict.items():
    ranked = find_collisions(tokens, stems)
    df_coll = collisions_to_dataframe(ranked, topn=100,
stemmer_name=name)
    fp = os.path.join(out_dir, f"collisions_top100_{name}.csv")
    df_coll.to_csv(fp, index=False)
    all_collisions.append(df_coll)
    print(f"Saved top-100 collisions for {name} to {fp} (potential
over-stemming examples).")

# Print a small on-screen preview of collisions
print("\n=== SAMPLE COLLISIONS (Top 10 per stemmer) ===")
for dfc in all_collisions:
    print("\n--", dfc['stemmer'].iloc[0], "--")
    print(dfc.head(10).to_string(index=False))

# Hand-checked pairs to illustrate over/under stemming cleanly
porter = PorterStemmer()
snowball = SnowballStemmer(language="english")
lancaster = LancasterStemmer()
regexp = RegexpStemmer('ing$|s$|e$|able$', min=4)
stem_fns = {
    "porter": porter.stem,
    "snowball": snowball.stem,
    "lancaster": lancaster.stem,
    "regexp": regexp.stem,
}
inspect_examples(stem_fns)

```

```

# Short narrative Observations (print to console + save)
observations = []

# Vocabulary reductions
vocab_original = len(set(tokens))
for name, stems in stems_dict.items():
    reduction = 100.0 * (1 - (len(set(stems)) / vocab_original))
    observations.append(f"{name.capitalize():<10} vocab reduction vs.
original: {reduction:.2f}%")

observations += [
    "",
    "Notes:",
    "- Lancaster is the most aggressive; expect more collisions
(possible over-stemming).",
    "- Snowball is a refined version of Porter for English; typically
balanced and multilingual in general.",
    "- Regexp only strips patterns you specify; safe but limited (may
under-stem vs. algorithmic stemmers).",
    "- Over-stemming: different words collapsing to one stem (see
collisions tables).",
    "- Under-stemming: related words kept apart (see checked examples
like 'analysis' vs 'analyst').",
]
obs_text = "\n".join(observations)
print("\n=== OBSERVATIONS SUMMARY ===")
print(obs_text)
with open(os.path.join(out_dir, "observations.txt"), "w",
encoding="utf-8") as f:
    f.write(obs_text)

print(f"\nAll outputs saved under: {os.path.abspath(out_dir)}")
print("Done.")

if __name__ == "__main__":
    main()

```


Output/Result:

```

PS C:\Users\Dell\Downloads\VII SEM\HO-NLP\LAB\EXP-3> & C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/D
ownloads/VII SEM\HO-NLP\LAB\EXP-3/EXP-3.py"
[nltk_data] Downloading package gutenberg to
[nltk_data]   C:\Users\Dell\nltk_data...
[nltk_data]   Unzipping corpora\gutenberg.zip.
[nltk_data] Downloading package brown to C:\Users\Dell\nltk_data...
[nltk_data]   Unzipping corpora\brown.zip.
[nltk_data] Downloading package reuters to C:\Users\Dell\nltk_data...
Building large corpus from Gutenberg + Brown + Reuters...
Preprocessing (lowercase, alphabetic only, remove stopwords, len>2)...
Applying stemmers...

=== SIZE SUMMARY ===
Total tokens (after preprocessing): 2,365,734
Unique tokens BEFORE stemming: 75,785
Stemmer Unique tokens AFTER Vocab reduction vs. original
porter 50446 33.44%
snowball 49772 34.32%
lancaster 41396 45.38%
regexp 64349 15.09%

Saved word-to-stems sample table to stemming_outputs\word_to_stems_5k.csv
Saved top-100 collisions for porter to stemming_outputs\collisions_top100_porter.csv (potential over-stemming examples).
Saved top-100 collisions for snowball to stemming_outputs\collisions_top100_snowball.csv (potential over-stemming examples).
Saved top-100 collisions for lancaster to stemming_outputs\collisions_top100_lancaster.csv (potential over-stemming examples).
Saved top-100 collisions for regexp to stemming_outputs\collisions_top100_regexp.csv (potential over-stemming examples).

=== SAMPLE COLLISIONS (Top 10 per stemmer) ===

-- porter --
stemmer stem distinct_original_words originals_sample

```

```

porter gener 22 general, generalities, generality, generalization, generalizations, generalize, g
eneralized, generalizing, generally, generals, generate, generated
porter commun 20 commun, communal, commune, communed, communes, communicate, communicated, communica
tes, communicating, communication, communicational, communications
porter inform 17 inform, informal, informality, informally, informant, informants, inform
ation, informational, informations, informative, informe, informed
porter equal 16 equal, equalized, equalizer, equalizers, equalizing, equall
on, equalize, equalized, equalizer, equalizers, equalizing, equall
porter oper 15 oper, operable, operant, operate, operated, operates, opera
ting, operation, operational, operationally, operations, operative
porter present 14 present, presentable, presentation, presentational, presentations, presente, pres
ented, presenter, presenting, presently, presentment, presentments
porter origin 14 origin, original, originality, originally, originals, originate, originat
ed, originates, originating, origination, originations, originator
porter observ 14 observ, observable, observance, observances, observant, observation, o
bservational, observations, observe, observed, observer, observers
porter determin 14 determin, determinability, determinable, determinant, determinants, determinate, determinatio
n, determinations, determinative, determine, determined, determines
porter liber 14 liber, liberal, liberalism, liberality, liberalization, liberalize, l
iberalized, liberalizing, liberally, liberals, liberate, liberated

-- snowball --
stemmer stem distinct_original_words originals_sample
snowball inform 17 inform, informal, informality, informally, informant, informants, inf
ormation, informational, informations, informative, informe, informed
snowball equal 16 equal, equalized, equalizing, equalities, equality, equaliz
ation, equalize, equalized, equalizer, equalizers, equalizing, equall
snowball determin 15 determin, determinability, determinable, determinant, determinants, determinate, determinatio
n, determinations, determinative, determine, determined, determinedly
snowball oper 15 oper, operable, operant, operate, operated, operates, op
erating, operation, operational, operationally, operations, operative

```

```

EXP-3
  stemming_outputs
    collisions_top100_lancast...
    collisions_top100_porter...
    collisions_top100_regexp...
    collisions_top100_snowb...
    observations.txt
    summary_metrics.csv
    word_to_stems_5k.csv
    EXP-3.py

OUTLINE
TIMELINE

snowball direct 14 direct, directed, directing, direction, directional, directionality, di
rectionally, directions, directive, directives, directivity, directly
snowball present 14 present, presentable, presentation, presentational, presentations, presente, p
resented, presenter, presenting, presently, presentment, presentations
snowball origin 14 origin, original, originality, originally, originals, originate, origi
nated, originates, originating, origination, originations, originator
snowball observ 14 observ, observable, observance, observances, observant, observation
, observational, observations, observe, observed, observer, observers
snowball wonder 14 wonder, wondered, wonderer, wonderful, wonderfully, wonderfulness, w
ondering, wonderingly, wonderings, wonderment, wonderments, wonderful
snowball liber 14 liber, liberal, liberalism, liberality, liberalization, liberalize
, liberalized, liberalizing, liberally, liberals, liberate, liberated

-- lancaster --
stemmer stem distinct_original_words originals_sample
lancaster cont 59 conc, conceal, concealed, concealing, concealment, concealments, conceals, concede,
conceded, concededly, concedes, conceding
lancaster fin 51 fin, fina, final, finale, finales, finalise, finalised, finalist, f
inalists, finality, finalization, finalize
lancaster com 45 com, come, comed, comedian, comedians, comeliness,
comely, comer, comers, comes, comi, comic
lancaster reg 45 reg, regal, regale, regaled, regaling, regan, regencies, regency, regenerat
e, regenerates, regenerating, regeneration
lancaster nat 42 nat, natal, nate, nation, national, nationale, nationalisation, nationalised, nationalism
, nationalisms, nationalist, nationalistic
lancaster man 41 man, mana, manage, manageable, managed, management, managements, manage
r, managerial, managers, manages, managing
lancaster adv 41 advance, advanced, advancement, advancements, advances, advancing, advantage, advantaged, advant
ageous, advantageously, advantages, advent
lancaster rat 39 rat, rata, ratable, rate, rateable, rated, rates, ratificati
on, ratifie, ratified, ratifiers, ratifies
lancaster min 38 min, mine, mineable, mined, miner, mineral, mineralization,
mineralized, minerals, miners, mines, mini
lancaster int 38 inc, incantation, incanted, incens, incentive, incentives, incise
, incur, incurable, incurably, incurs, int

-- regexp --
stemmer stem distinct_original_words originals_sample
regexp present 5 present, presentable, presente, presenting, presents
regexp suit 5 suit, suitable, suite, suiting, suits
regexp work 5 work, workable, worke, working, works
regexp speak 5 speak, speakable, speake, speaking, speaks
regexp pay 5 pay, payable, paye, paying, pays
regexp rat 5 rat, ratable, rate, rating, rats
regexp eat 5 eat, eatable, eate, eating, eats
regexp read 5 read, readable, reade, reading, reads
regexp answer 5 answer, answerable, answers, answering, answers
regexp discern 5 discern, discernable, discerne, discerning, discerns

=== CHECKED EXAMPLES (POTENTIAL OVER-STEMMING) ===
universe / university | porter: univers / unvers | snowball: univers / univers | lancaster: univers / univers | regexp: univers /
university
policy / police | porter: polici / polic | snowball: polici / polic | lancaster: policy / pol | regexp: policy / polic
general / generation | porter: gener / gener | snowball: general / generat | lancaster: gen / gen | regexp: general / generation
final / finally | porter: final / final | snowball: final / final | lancaster: fin / fin | regexp: final / finally
organ / organic | porter: organ / organ | snowball: organ / organ | lancaster: org / org | regexp: organ / organic

=== CHECKED EXAMPLES (POTENTIAL UNDER-STEMMING) ===
analysis / analyst | porter: analysi / analyst | snowball: analysis / analyst | lancaster: analys / analyst | regexp: analysi /
analyst
connect / connection | porter: connect / connect | snowball: connect / connect | lancaster: connect / connect | regexp: connect /
connection
biology / biological | porter: biolog / biolog | snowball: biolog / biolog | lancaster: biolog / biolog | regexp: biology / biolo
gical
economy / economic | porter: economi / econom | snowball: economi / econom | lancaster: econom / econom | regexp: economy / eco

```

```

EXP-3
  stemming_outputs
    collisions_top100_lancast...
    collisions_top100_porter...
    collisions_top100_regexp...
    collisions_top100_snowb...
    observations.txt
    summary_metrics.csv
    word_to_stems_5k.csv
    EXP-3.py

OUTLINE
TIMELINE

mineralized, minerals, miners, mines, mini
lancaster int 38 inc, incantation, incanted, incens, incentive, incentives, incise
, incur, incurable, incurably, incurs, int

-- regexp --
stemmer stem distinct_original_words originals_sample
regexp present 5 present, presentable, presente, presenting, presents
regexp suit 5 suit, suitable, suite, suiting, suits
regexp work 5 work, workable, worke, working, works
regexp speak 5 speak, speakable, speake, speaking, speaks
regexp pay 5 pay, payable, paye, paying, pays
regexp rat 5 rat, ratable, rate, rating, rats
regexp eat 5 eat, eatable, eate, eating, eats
regexp read 5 read, readable, reade, reading, reads
regexp answer 5 answer, answerable, answers, answering, answers
regexp discern 5 discern, discernable, discerne, discerning, discerns

=== CHECKED EXAMPLES (POTENTIAL OVER-STEMMING) ===
universe / university | porter: univers / unvers | snowball: univers / univers | lancaster: univers / univers | regexp: univers /
university
policy / police | porter: polici / polic | snowball: polici / polic | lancaster: policy / pol | regexp: policy / polic
general / generation | porter: gener / gener | snowball: general / generat | lancaster: gen / gen | regexp: general / generation
final / finally | porter: final / final | snowball: final / final | lancaster: fin / fin | regexp: final / finally
organ / organic | porter: organ / organ | snowball: organ / organ | lancaster: org / org | regexp: organ / organic

=== CHECKED EXAMPLES (POTENTIAL UNDER-STEMMING) ===
analysis / analyst | porter: analysi / analyst | snowball: analysis / analyst | lancaster: analys / analyst | regexp: analysi /
analyst
connect / connection | porter: connect / connect | snowball: connect / connect | lancaster: connect / connect | regexp: connect /
connection
biology / biological | porter: biolog / biolog | snowball: biolog / biolog | lancaster: biolog / biolog | regexp: biology / biolo
gical
economy / economic | porter: economi / econom | snowball: economi / econom | lancaster: econom / econom | regexp: economy / eco

```

```

nomic
happy      / happiness | porter: happi / happi | snowball: happi / happi | lancaster: happy / happy | regexp: happy / happines

=== OBSERVATIONS SUMMARY ===
Porter  vocab reduction vs. original: 33.44%
Snowball vocab reduction vs. original: 34.32%
Lancaster vocab reduction vs. original: 45.38%
Regexp  vocab reduction vs. original: 15.09%

Notes:
- Lancaster is the most aggressive; expect more collisions (possible over-stemming).
- Snowball is a refined version of Porter for English; typically balanced and multilingual in general.
- Regexp only strips patterns you specify; safe but limited (may under-stem vs. algorithmic stemmers).
- Over-stemming: different words collapsing to one stem (see collisions tables).
- Under-stemming: related words kept apart (see checked examples like 'analysis' vs 'analyst').

All outputs saved under: C:\Users\Dell\Downloads\VII SEM\HO-NLP\LAB\EXP-3\stemming_outputs
Done.
PS C:\Users\Dell\Downloads\VII SEM\HO-NLP\LAB\EXP-3>

```

Post Lab Question-Answers:

1. Explain Under-Stemming and Over-Stemming errors with the help of suitable examples.

Ans: i) Under-Stemming Error

- This occurs when the stemmer fails to reduce related words to the same root form — it “under-stems” and keeps them separate.
- Example:
 - Words: connect, connected, connection
 - If the stemmer outputs:
 connect → connect
 connected → connect
 connection → connection (❌ different stem)
 - Here, connection is not reduced to connect, so related words are treated as different.

ii) Over-Stemming Error

- This occurs when the stemmer reduces unrelated words to the same root form — it “over-stems” and merges them incorrectly.
- Example:
 - Words: universe, university
 - If the stemmer outputs:
 universe → univers
 university → univers (❌ unrelated words now have same stem)
 - Here, two different concepts are mistakenly grouped as the same.

Outcomes: CO2 – Comprehend Words and Word Forms in NLP

Conclusion (based on the Results and Outcomes achieved):

In conclusion, the experiment demonstrated that stemming is a crucial step in the NLP pipeline for reducing words to their base or root form, thereby simplifying text analysis and reducing vocabulary size. The results showed that each stemmer has its own strengths and weaknesses — Porter is fast and simple, Snowball is more versatile with better accuracy and multilingual support, Lancaster is highly aggressive but prone to over-stemming, and Regexp offers flexibility through customizable rules. The effectiveness of a stemmer depends on the application requirements, as over-stemming can merge unrelated words while under-stemming can fail to unify related terms. Choosing the right stemmer is therefore essential for balancing accuracy and dimensionality reduction in NLP tasks.

References:

Books/ Journals/ Websites:

1. Allen James, Natural Language Understanding, Benjamin Cumming, Second Edition, 1995
 2. Jurafsky, Dan and Martin, James, Speech and Language Processing, Prentice Hall, 2008
 3. Palash Goyal, Karan Jain, Sumit Pandey, Deep Learning for Natural Language Processing: Creating Neural Networks with Python, Apress, 2018
-